计算机领域本科教育教学改革试点
工作计划（“101计划”）研究成果

数据结构

授课教师：谭光华

湖南大学　信息科学与工程学院

第 6 章

优先级队列

提纲

6.1　问题引入

6.2　优先级队列的定义

6.3　二叉堆

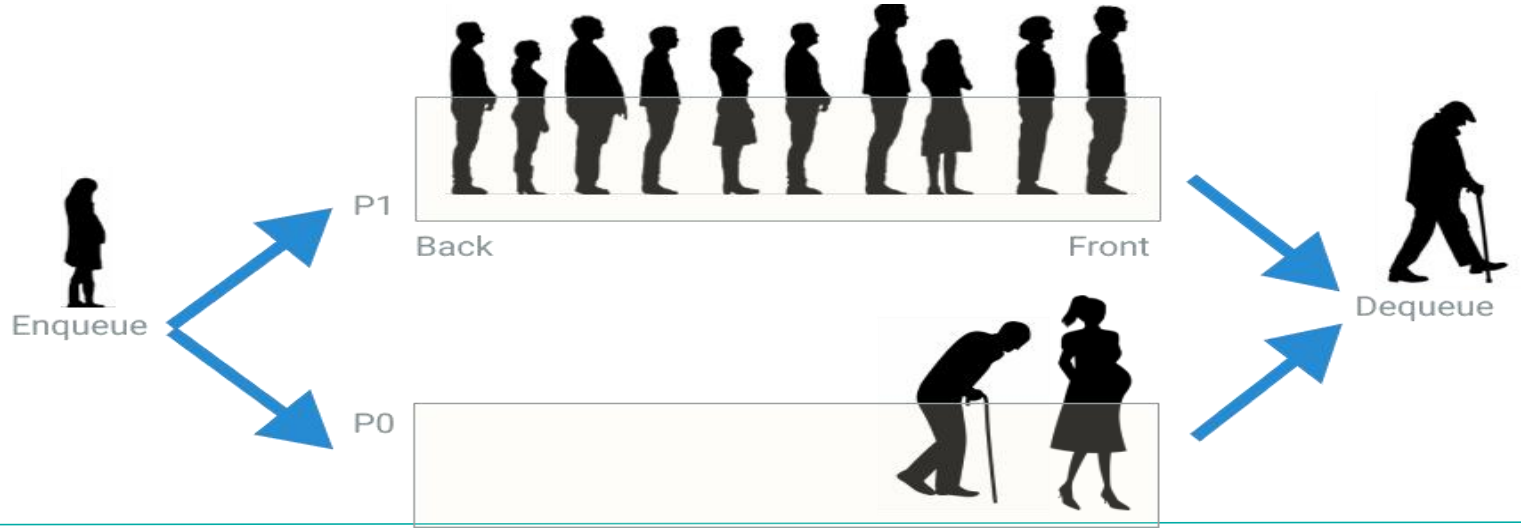
6.4　多叉堆

6.6　优先级队列应用

6.8　应用场景

6.1 问题引入：带优先级的服务处理

- **问题**
  - 第3章介绍的**队列**能够按**先到先得**的方式来处理，但实际问题中（例如医院或者银行）还有可能需要考虑加急的情况，更一般地说就是**优先级**。如何按优先级进行处理?
- **关键：** 出队的顺序需要由元素本身的**优先级**来决定，而不是进队的顺序



6.2 优先级队列的定义

- **优先级队列（Priority Queue）**
  - **是一种特殊的队列。与普通的队列相比**
    - **相同点：都支持出队和进队的操作**
    - **不同点：优先级队列的出队顺序按事先规定的优先级顺序进行**

ADT PriorityQueue {
数据对象：
　元素取自全集 *U* 的可重集合 *E*，表示优先级队列中包含的元素。
数据关系：
　全集 *U* 中的元素须满足严格弱序。
基本操作：
　（省略初始化、销毁、清除内容、判断为空、查询元素个数等操作）
　Insert(*pq*, *x*)：在优先级队列 *pq* 中插入元素 *x*。
　ExtractMin(*pq*)：从优先级队列 *pq* 中删除优先级最高（也就是值最小）的元素，并返回。
　PeekMin(*pq*)：返回优先级队列 *pq* 中优先级最高的元素（元素仍然保留在优先级队列中）。}

优先级队列的实现

- **优先级队列的实现**
  - **线性表**
    - **使用线性表实现时出队操作需要遍历当前优先级队列中的所有元素，从而找到优先级最高的元素。**
    - **出队的时间复杂度O(n)，效率较低**
  - **二叉堆**
    - **一种常见的堆，常用来实现优先级队列**
    - **二叉堆最早由 J. W. J. Williams 于 1964 年提出，作为支持堆排序的一种数据结构**

6.3 二叉堆

- **二叉堆的定义**
  - 父结点元素和子结点元素满足一定大小关系的**完全二叉树**
  - 根据条件不同，可分为最小堆和最大堆
- **最小堆**
  - 如果完全二叉树 T 中所有父结点的元素均**小于等于（不大于）**子结点的元素，则称 T 为最小堆
- **最大堆**
  - 如果完全二叉树 T 中所有父结点的元素均**大于等于（不小于）**子结点的元素，则称 T 为最大堆

二叉堆

- **最小堆和最大堆的区别只在于父子结点元素之间的大小关系**

最小堆的数组表示：

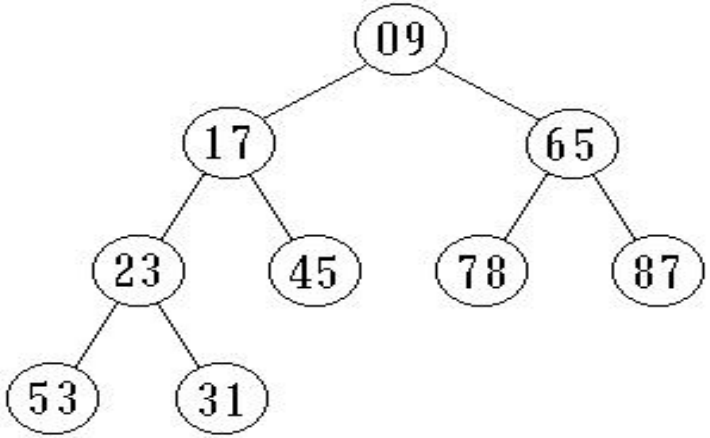
$K_i \le K_{2i}$ && $K_i \le K_{2i+1}$

堆顶元素为最小值

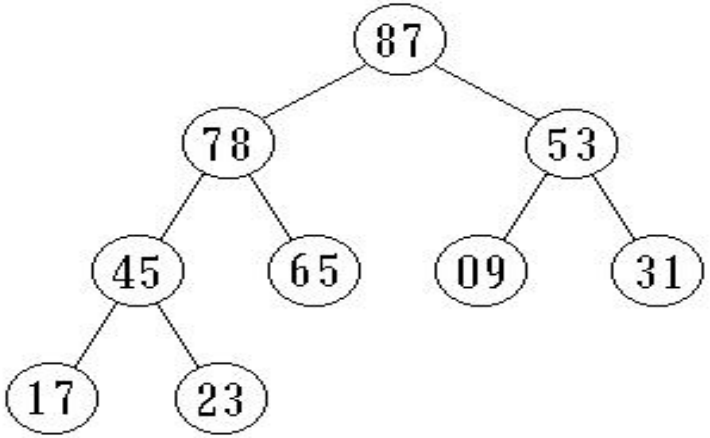
最大堆的数组表示：

$K_i \ge K_{2i}$ && $K_i \ge K_{2i+1}$

堆顶元素为最大值



（a）最小堆

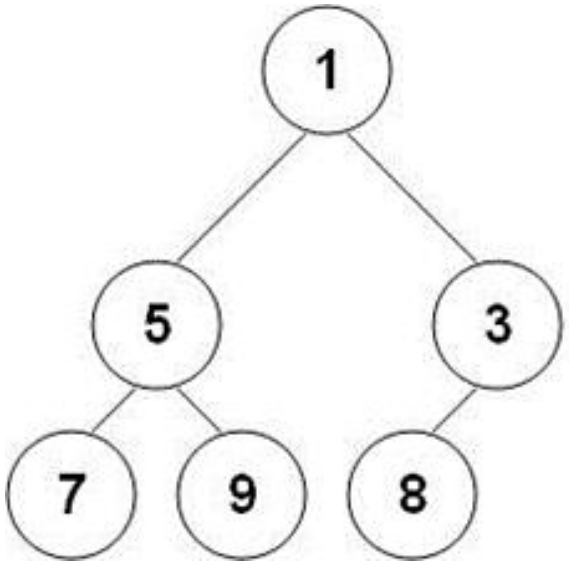


（b）最大堆

本章中的二叉堆都以**最小堆**为例进行讲解，最大堆的情况可类推得到

二叉堆的表示

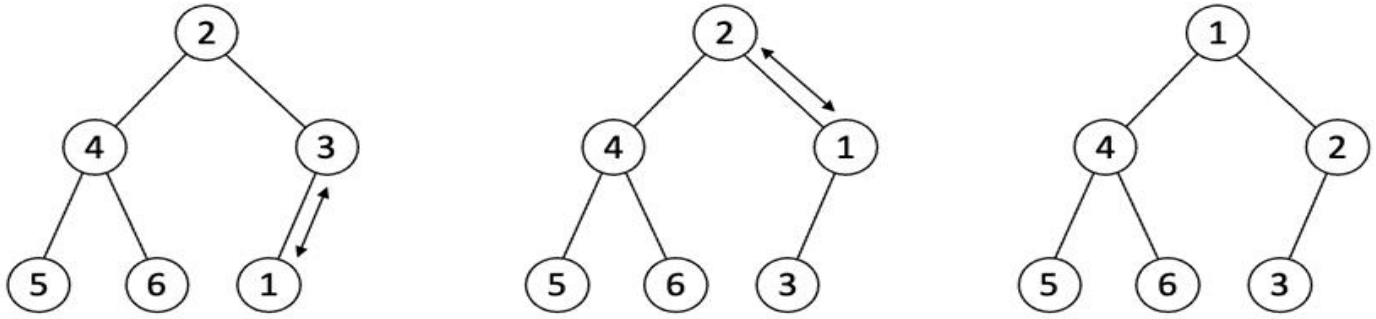
- **二叉堆是一棵完全二叉树，可用顺序表表示**
  - **结点 i 的左右子结点（如果存在）的下标分别为 2i 和 2i+1**
  - **结点 i 的父节点（如果存在）的下标为**$\lfloor i/2 \rfloor$



结点	1	5	3	7	9	8
下标	1	2	3	4	5	6

6.3 二叉堆

- **主要操作：插入、删除、建堆**
  - **这些操作的实现均基于两个基本操作：上调和下调；**
- **上调**
  - **如果堆中某结点 i 小于其父结点 p，交换结点 i 和结点 p 的元素**
  - **再看新的父结点与它的大小关系。重复该过程，直到结点 i 被调到根结点位置或者和新的父结点大小关系满足条件。**



(1) (2) (3)

二叉堆的上调操作

- **上调（SiftUp）**
  - **SiftUp 先将结点 i 的元素保存在临时变量中，随着调整将父结点的元素往“下”移动，最后再将原来结点 i 的元素填入合适的位置，从而减少赋值操作的次数**

算法6-1： 二叉堆的上调操作 SiftUp(h, i)

输入： 堆 h 和上调起始位置 i

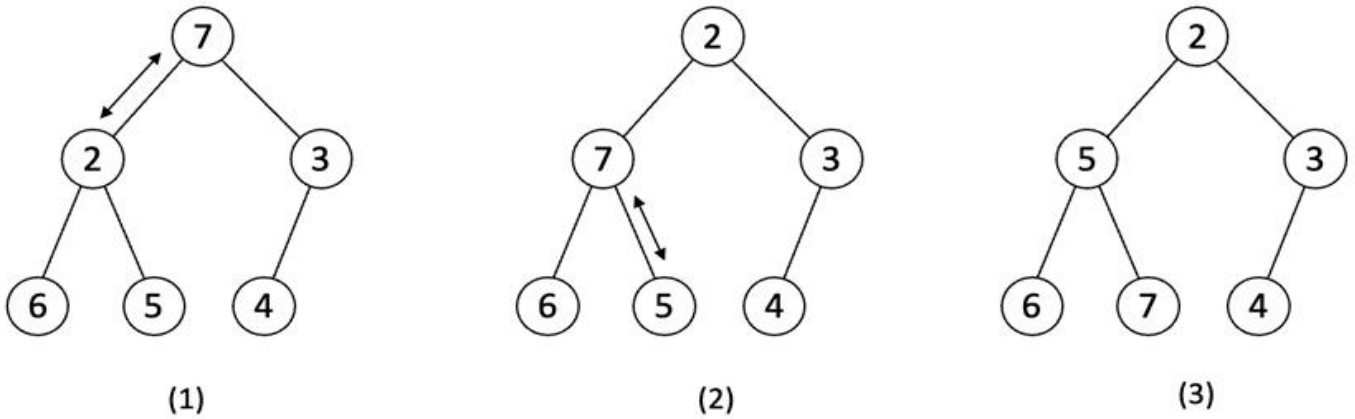
输出： 上调后满足堆性质的h

```
elem ← h.data[i]
while i > 1 且 elem < h[i / 2] do  //当前结点小于其父结点
  | h.data[i] ← h.data[i / 2]  //将i的父结点元素下移
  | i ← i / 2  //i指向原结点的父结点，即向上调整
end
h.data[i] ← elem
```

循环的次数不超过树的高度，因此时间复杂度为O(log n)

二叉堆的下调操作

- **下调（SiftDown）**
  - **如果堆中某结点 i 大于其子结点，交换结点 i 和子节点中具有较小值的结点**
  - **重复该过程，直到结点 i 被调到叶结点位置或者和新的子结点大小关系满足条件**



二叉堆的下调操作

算法6-2： 二叉堆的下调操作 SiftDown(h, i)

输入： 堆 h 和下调起始位置 i

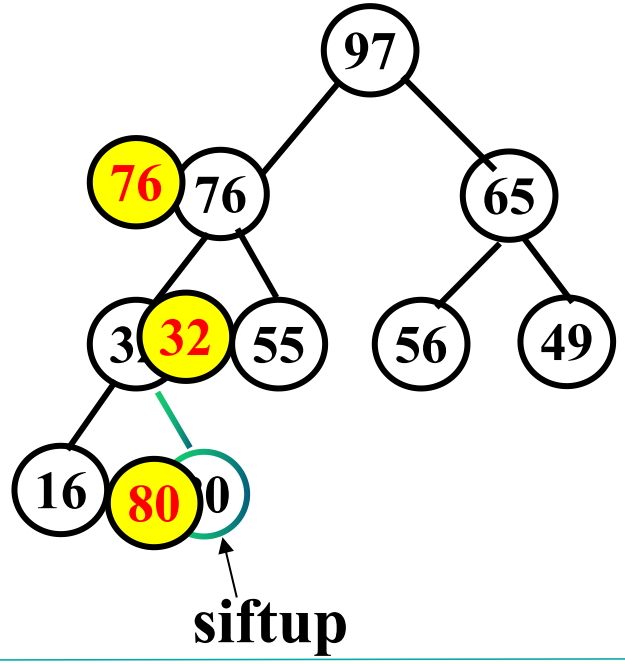
输出： 下调后满足堆性质的h

```
last ← h.size  //这是最后一个元素的位置
elem ← h.data[i]
while true do
  | child ← 2i  //child当前是i的左孩子的位置
  | if child<last 且h.data[child+1]<h.data[child] then //如果i有右孩子并且右孩子更小
  | | child ← child + 1  //child更新为i的右孩子的位置
  | else if child>last  //如果i是叶子结点
  | | break  //已经调整到底，跳出循环
  | end
  | if h.data[child] < elem then  //若较小的孩子比elem小
  | | h.data[i] ← h.data[child]  //将较小的孩子结点上移
  | | i ← child  //i指向原结点的孩子结点，即向下调整
  | else  //若所有孩子都不比elem小
  | | break  //则找到了elem的最终位置，跳出循环
  | end
end
h.data[i] ← elem
```

循环的次数不超过树的高度，因此时间复杂度为O(log n)

二叉堆的插入操作

- **插入**
  - **将待插入元素放在堆尾（为什么插在堆尾？）**
    - 不影响原来的堆结构，只有新插入的结点可能违反最小堆的性质
  - 将堆尾的元素通过SiftUp往上调到正确的位置



最大堆

二叉堆的插入操作

- **插入**
  - **将待插入元素放在堆尾**
  - **将堆尾的元素通过SiftUp往上调到正确的位置**

算法6-3： 二叉堆的插入操作 Insert(h, x)

输入： 堆 h 和待插入元素 x

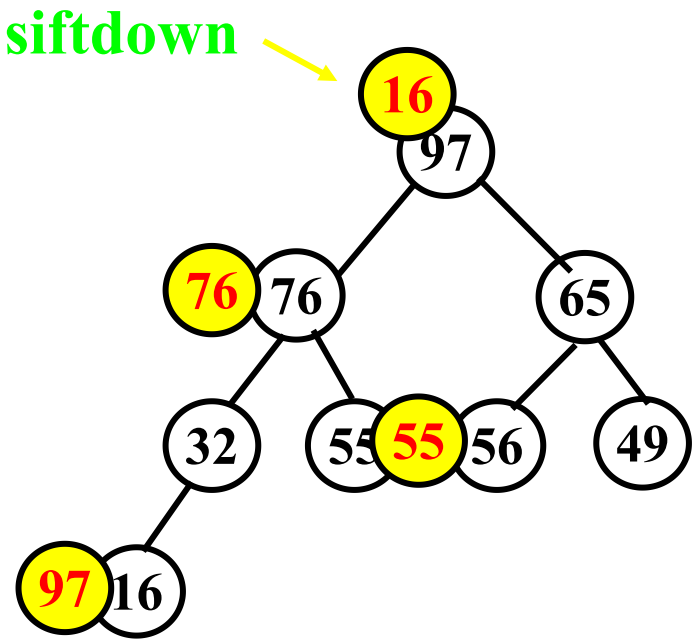
输出： 将元素插入后的堆

1. $h.size \leftarrow h.size + 1$
2. $last \leftarrow h.size$
3. $h.data[last] \leftarrow x$ //暂时将x放入最后一个元素的位置
4. SiftUp(h, $last$) //从最后一个位置上调

时间复杂度取决于SiftUp的时间复杂度，为O(log n)

二叉堆的删除操作

- **删除：删除堆顶最小的元素（ExtractMin）**
  - 将**堆尾的元素与堆顶最小的元素**进行交换（为什么？）
    - 不影响原来堆的结构，最多只有堆顶的元素违反最小堆的性质
  - 将堆顶的元素通过SiftDown下调到合适的位置



最大堆

二叉堆的删除操作

- **删除：删除堆顶最小的元素（ExtractMin）**
  - **将堆尾的元素与堆顶最小的元素进行交换**
  - **将堆顶的元素通过SiftDown下调到合适的位置**

算法6-4： 二叉堆的删顶操作 ExtractMin(h)

输入： 堆 h

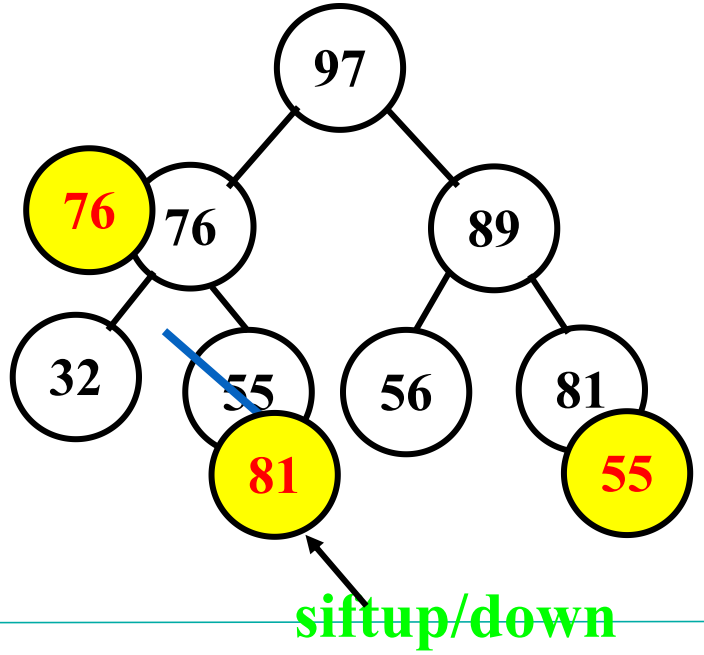
输出： h中的最小元，以及删除了最小元素后的堆h

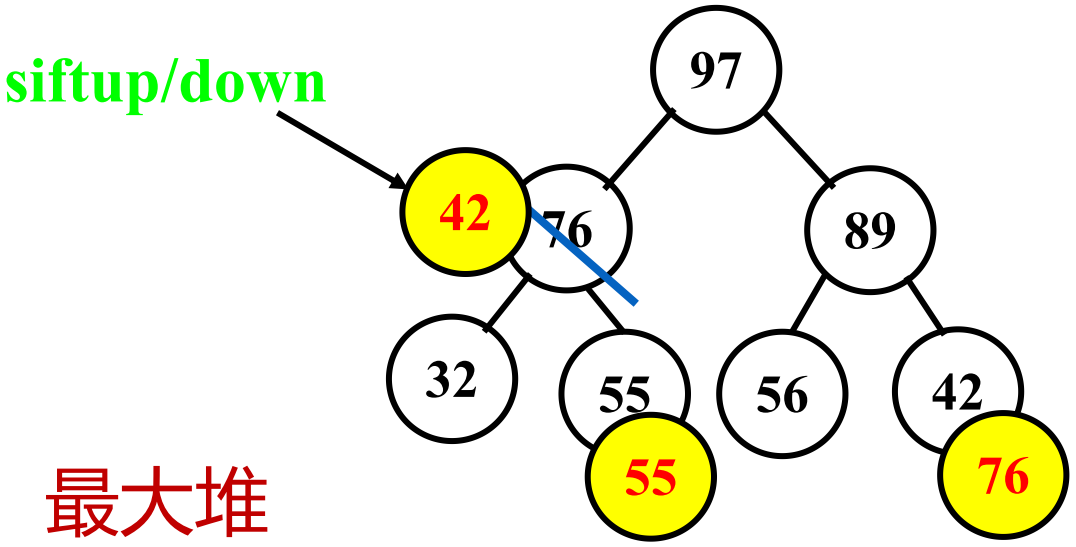
1. $min_key \leftarrow h.data[1]$ //这是将要返回的最小元
2. $last \leftarrow h.size$ //这是删除前最后一个元素的位置
3. $h.size \leftarrow h.size - 1$
4. $h.data[1] \leftarrow h.data[last]$ //暂时将删除前最后一个元素放入根的位置
5. SiftDown(h, 1) //从根结点下调
6. **return** min_key

时间复杂度取决于SiftDown的时间复杂度，为O(log n)

二叉堆的删除操作

- **删除：删除堆中的任意元素**
  - 将**待删除结点与最后一个结点互换**
  - 先对互换后该位置的结点进行siftup
  - 不可移动时，再进行siftdown

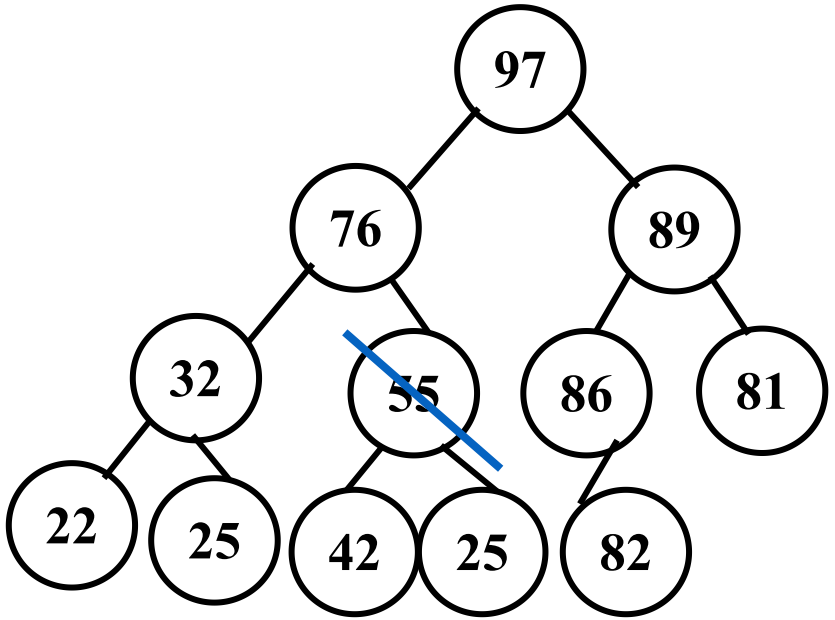




最大堆

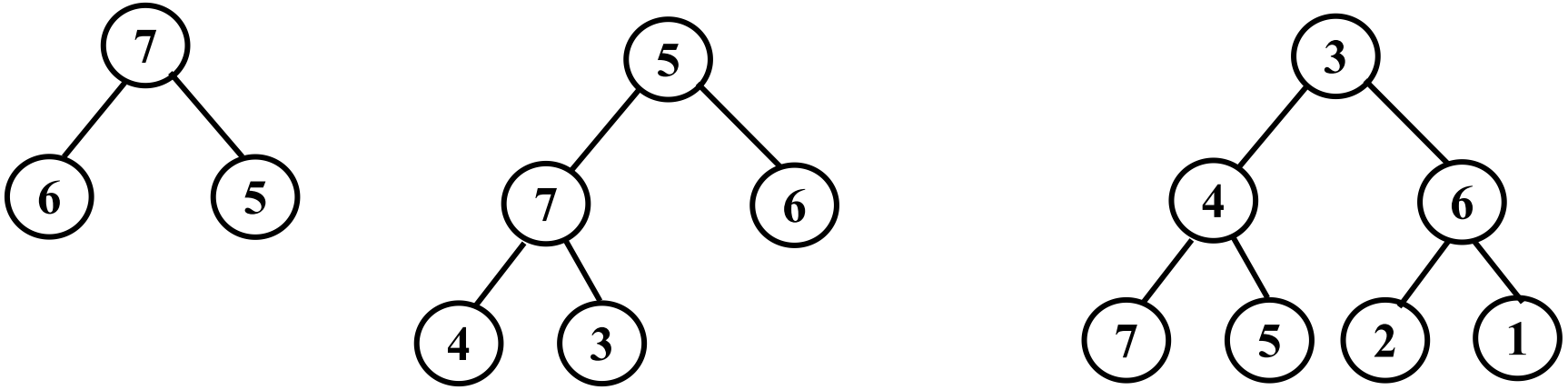
思考

- **删除的过程中，上推和下推可能同时发生吗，什么时候发生上推?**
  - **不可能同时发生上推和下推**
    - **一旦发生上推，说明需与父节点交换**
    - **此时，父节点大于所有子节点，无需下推**
  - **待删除节点的父节点与最后一个节点不处于同一棵子树时，可能产生上推**



二叉堆的建堆操作

- **朴素建堆操作**
  - **通过将n个元素依次插入堆中实现建堆**
    - **相当于依次上调数组中每个元素**



构建最小堆：7，6，5，4，3，2，1

二叉堆的建堆操作

- **朴素建堆操作(MakeHeapUp)**
  - **通过将n个元素依次插入堆中实现建堆**
    - **相当于依次上调数组中每个元素**

算法6-5： 二叉堆的朴素建堆操作 MakeHeapUp(h)

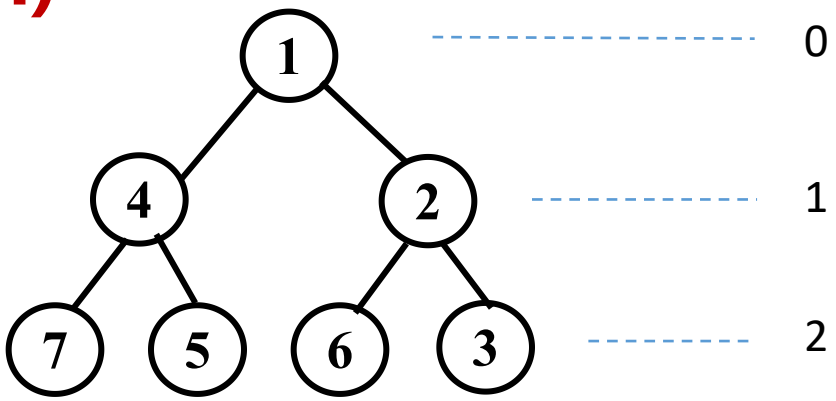
输入： 存储在 h 中的数据

输出： 满足堆性质的堆h

```
last ← h.size  //这是最后一个元素的位置
for i ← 2 to last do
| SiftUp(h, i)
end
```

二叉堆的建堆操作

- **时间复杂度分析**
  - 最后一层的元素最多可达n/2
  - 最后一层的元素上调的次数最多可达树的高度logn
  - 因此，朴素建堆的时间复杂度为O(nlogn)
- **也可通过数学推导得到：**
  - 第i层的结点最多上调的次数为i
  - 第i层的结点数为2^i
  - n个结点的总移动次数为：



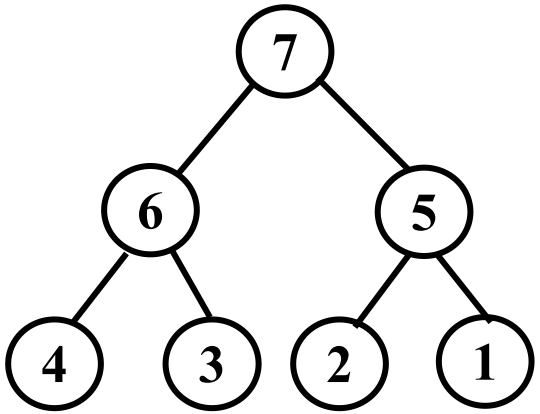
$$\sum_{i=0}^{\log n} i2^i = (\log n - 1)2n + 2 = 2n\log n - 2n + 2$$

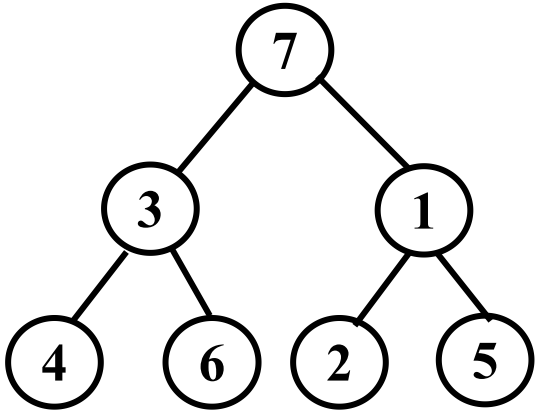
二叉堆的快速建堆操作

- **快速建堆**
  - 朴素建堆操作通过逐个元素**上调**的方式建堆，底层结点较多，上调的次数也较多，因此时间复杂度高
    - 约一半的节点位于叶节点，而这些节点需要调整的次数最多
  - 也可通过逐个元素**下调**的方式建堆，上层结点的下调次数较多，但是上层结点数目少，可以降低时间复杂度
    - 叶子结点不用调整（无孩子节点，满足堆的定义）
    - 每往上一层，需要调整的最大次数+1，但节点数减半

二叉堆的快速建堆操作

- 从最后一个非叶子节点开始，从上往下调整
  - 构建最小堆：7, 6, 5, 4, 3, 2, 1





二叉堆的快速建堆操作

- **快速建堆（MakeHeapDown）**
  - **从最后一个非叶子节点开始，从上往下调整**

算法6-6： 二叉堆的快速建堆操作 MakeHeapDown(h)

输入： 存储在 h 中的数据

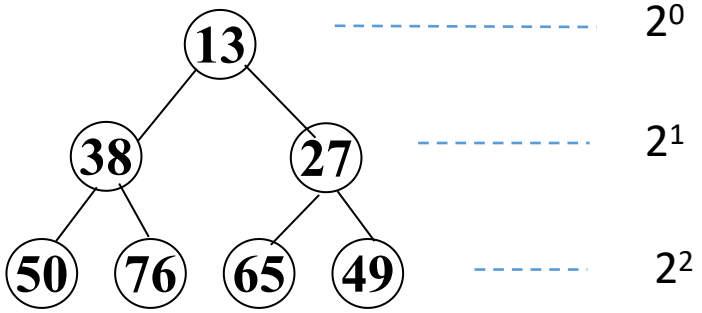
输出： 满足堆性质的堆h

```
last ← h.size  //这是最后一个元素的位置
for i ← last/2 downto 1 do  // last/2是最后一个元素的父结点的位置
  | SiftDown(h, i)
end
```

二叉堆的快速建堆操作

- **时间复杂度分析：**
  - **有将近一半的结点（叶子结点）无需调整，因此更加高效**
  - **1/4的结点最多能向下移动一层**
  - **树中每向上一层，节点数目为前一层的一半，元素移动的次数+1**
  - **总的移动次数为：**

$$\sum_{i=1}^{\log n}(i-1)\frac{n}{2^i}=\frac{n}{2}\sum_{i=1}^{\log n}\frac{i-1}{2^{i-1}}\approx O(n)$$



总的结点个数：2^3-1=7

快速建堆操作的时间复杂度为O(n)

6.4 多叉堆

- **多叉堆，也称为d堆（d-ary heap 或 d heap）**
  - **由 Donald B. Johnson 于 1975 年提出**
  - **多叉堆的每个结点有d个子节点（二叉堆2个子节点）**
  - **多叉堆是一棵完全多叉树，可以用数组来保存堆中的元素**
    - **元素 0 是根结点，元素 1~d 是根结点的子结点，而紧接着的 d^2 个元素是根结点的孙子结点，以此类推**
    - **结点 i 的 d 个子结点的下标分别为 d*i+j，其中1⩽ j⩽ d**
    - **结点 i 的父节点的下标为$\lfloor (i-1)/d \rfloor$**

多叉堆的上调操作

- **上调**
  - **扩展二叉堆的SiftUp操作**

算法6-7： 多叉堆的上调操作 SiftUpD(h, d, i)

输入： 堆 h、堆的分叉数 d 和上调起始位置 i

输出： 上调后满足d叉堆性质的h

```
elem ← h.data[i]
while i > 0 and elem < h.data[(i−1) / d] do
  | h.data[i] ← h.data[(i−1) / d]        //与父节点交换
  | i ← (i−1) / d
end
h.data[i] ← elem
```

时间复杂度依赖于堆的高度，为 $\log_d n$

多叉堆的下调操作

算法6-8：多叉堆的下调操作 SiftDownD(*h, d, i*)
输入：堆 *h*、堆的分叉数 *d* 和下调起始位置 *i*
输出：将堆 *h* 中的元素下调以满足堆性质

```
last ← h.size - 1  //这是最后一个元素的位置
elem ← h.data[i]
while true do
  | child ← i
  | for k ← 1 to d do  //找所有孩子中最小的
  | | if d×i+k ≤ last 且h.data[d×i+k]<h.data[child] then
  | | | child ← d×i+k  //child更新为更小的孩子的位置
  | | end
  | end
  | if child = i then  //前面for循环未执行，i是叶结点
  | | break  //已经调整到底，跳出循环
  | end
  | if h.data[child]<elem then  //若最小的孩子比elem小
  | | h.data[i] ← h.data[child]  //将最小的孩子结点上移
  | | i ← child  //i指向原结点的孩子结点，即向下调整
  | else  //若所有孩子都不比elem小
  | | break  //则找到了elem的最终位置，跳出循环
  | end
end
h.data[i] ← elem
```

查找最小孩子结点

与最小孩子结点交换

时间复杂度依赖于堆的高度及d个孩子结点中查找最小值的复杂度，为$d\log_d n$

多叉堆的分析

- **时间复杂度分析**
  - **d叉堆的上推操作的复杂度为** $O(\log_d n)$，**下推操作的复杂度为** $O(d\log_d n)$
    - **相同结点数的多叉堆高度更小，因此上调操作的性能会更好**
    - **下调操作由于要遍历d个孩子结点，性能会变差**
  - **因此，多叉堆可用于上调操作比下调操作频繁的应用场景**
  - **与二叉堆相比，多叉堆有更好的高速缓存特性，因此实际使用中能够在现代计算机系统结构上取得更好的运行效率**

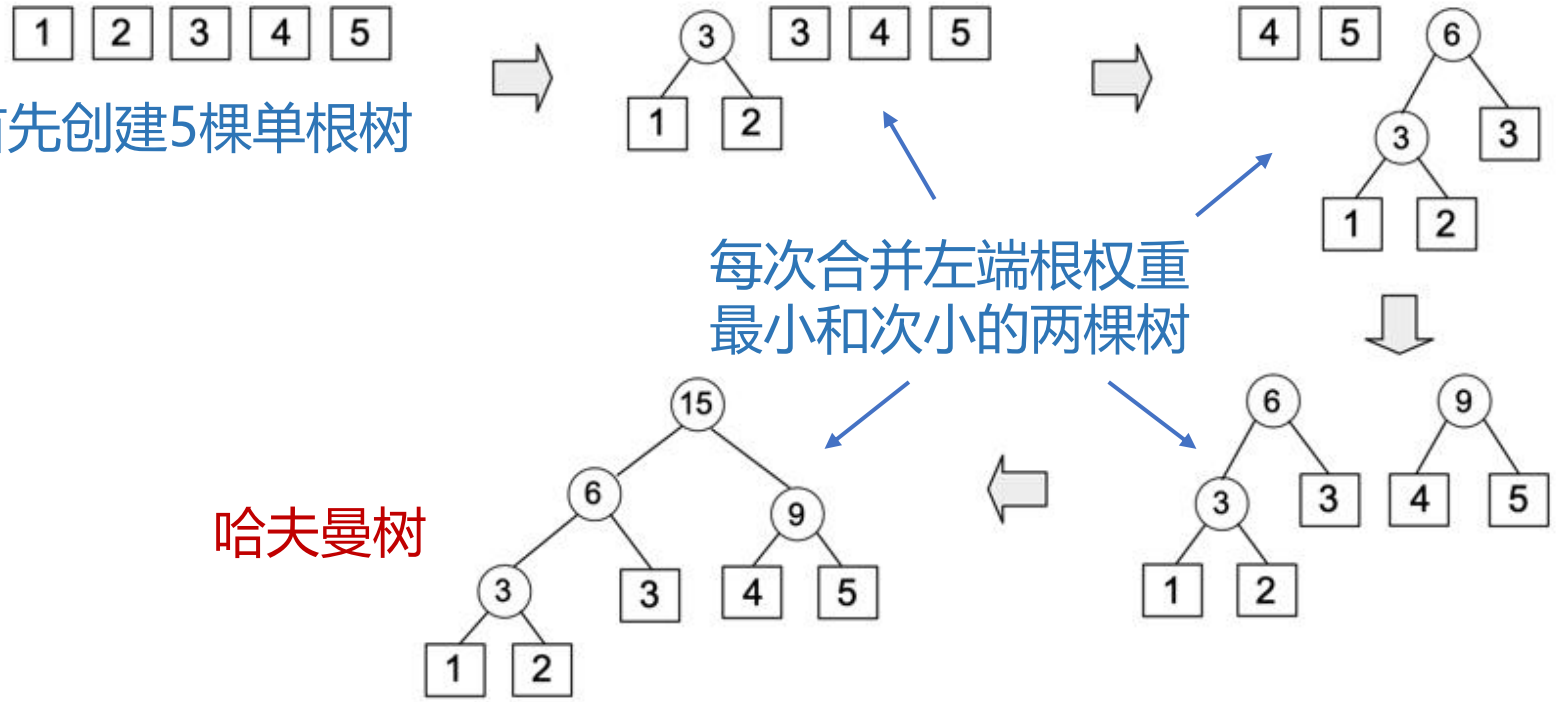
6.6 优先级队列应用：哈夫曼树的构建

- **哈夫曼树构建**

初始建堆：O(n)

每次迭代需2次删除堆顶元素+1次合并后插入的操作
迭代n-1次

最优二叉树的构建过程



算法5-15：创建哈夫曼树 CreateHuffmanTree(*w*)

输入： 权重值的数据集w，$|w| \geq 2$
输出： 哈夫曼树

```
tree_set ← ∅  //二叉树集合的初始化
n ← Length(w)        //n个权重
for i ← 1 to n do      //初始化n个二叉树
| tree ← new BinaryTreeNode( ) //创建叶结点
| tree.left  ← NIL
| tree.right ← NIL
| tree.weight ← Extract(w)    //从数据集w中取出一个值，作为结点权重
| Insert(tree_set, tree)                //将单结点二叉树放入集合tree_set
end
|     //合并二叉树
```

- 时间$T_{W_Extract}$
- 时间T_{Q_Insert}

算法5-15：创建哈夫曼树 CreateHuffmanTree(*w*)

```
|
for i ← 1 to n-1 do      //合并二叉树，共n-1次
| tree ← new BinaryTreeNode( )      //新建树根结点
| tree.left ← ExtractMin(tree_set)      //取出根权重最小树作为左子树
| tree.right ← ExtractMin(tree_set)      //取出根权重次小树作为右子树
| tree.weight ← tree.left.weight + tree.right.weight  //设置新树的根权重
| Insert(tree_set, tree)                              //将新树插入集合tree_set
end
tree ← Extract(tree_set)      //取出集合中唯一的二叉树，即哈夫曼树
return tree
```

- 时间$T_{Q_ExtractMin}$
- 时间$T_{Q_ExtractMin}$
- 时间T_{Q_Insert}
- 时间$T_{Q_Extract}$

6.6 优先级队列应用：哈夫曼树的构建

- **哈夫曼树构建**
  - 持续地从一个二叉树集合中取出带权路径**长度最小和次小**的两棵二叉树，并把**合并后的树加回到集合**里
  - 可以利用**二叉堆**来保存权值，分别使用**ExtractMin和Insert**来完成从集合取出最小、次小以及加回到集合的操作
    - 二叉堆的插入和删除元素操作都是O(logn)的时间复杂度
    - 建堆的时间复杂度是O(n)
    - 整个算法在建堆之后一共要进行2n-2次删除和n-1次插入操作，因此总的复杂度为O(n log n)

6.8 应用场景：离散事件模拟

- **离散事件模拟**
  - 离散事件系统中，所有状态只在特定的、离散的时间点发生变化，而这些变化都可被抽象为一系列定义好的事件。
  - 离散事件模拟被广泛应用于各种领域，如工业制造、交通系统、医疗保健、金融等。
- **三个重要组成部分：时钟、状态和事件列表**
  - 时钟：表示系统当前的模拟进度，它可以是现实时间，也可以只是逻辑概念
  - 状态：反映了整个系统的所有需要关注的属性
  - 事件：事件列表中包含了所有将来要发生的事件和它们会发生的时间点

6.8 应用场景：离散事件模拟

- **离散事件模拟**
  - 模拟器从事件列表中取出所有当前时刻发生的事件，并根据这些事件对应修改系统的状态，向事件列表中增加新的（会在将来发生的）事件。
  - **优先级队列**非常适合用来维护事件列表
    - 以“事件发生的时间点”作为元素的优先级维护队列，并不断出队、处理事件，直到事件列表为空，或者达到了预定的模拟时间点即可停止。

小结

- **优先级队列**是一种常用的数据结构，提供了比栈和队列更加丰富的处理顺序，可用于很多真实场景。
- **堆**是优先级队列的一类实现方式，由基本的堆性质衍生出了多种不同的堆的设计，它们在堆的基本操作上具有不同的复杂度。
- 这其中，**二叉堆**是一种隐式数据结构，它将元素的逻辑结构蕴含在存储结构中，避免了额外的指针域空间开销，实现起来也比较简洁，因此得到了广泛的应用。